CIS 613: Midterm Review

1

Verification vs Validation

- **Verification**: A set of activities that ensure that the software conforms to its specifications:

  "Are we building the thing *right*?"

- **Validation**: A set of activities that ensure that the software built meets the customer's *needs and expectations*.

  "Are we building the *right* thing?"

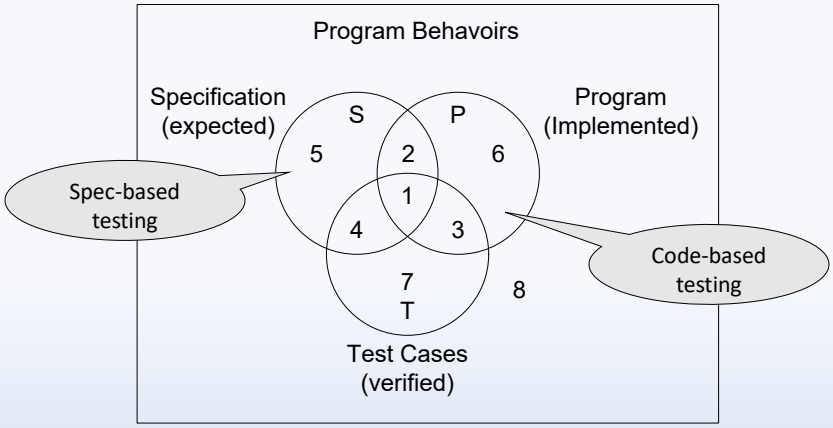
Verification and Validation (V&V) should establish confidence that the software is "fit for purpose"

- This does *not* mean the software is *completely* free of defects.
- It *does mean* the software must be good enough for its intended use, where that use determines the degree of confidence that is needed.

This class focuses on ***verification*** based on correct specifications.

2

Testing and Program Behaviors



3

Introduction

- Spec-Based vs Code Based Testing
- Psychology of testing – Destructive in nature
- Objective of testing – “to find the largest number of errors in the least amount of time with a minimum allocation of resources”
- Dynamic vs Static Verification
- False Positives vs False Negatives
- Levels of Testing

4

Definitions

- **Error** – People make errors. A good synonym is a *mistake*.
- **Fault** – The result of an **error**, regardless of if it has ever occurred. May also be called a *defect* or *bug*.
- **Failure** – Occurs when code corresponding to a **fault** executes.
- **Incident** – A symptom of a **failure** that is noticeable to the user.
- **Test Case** – A set of inputs and outputs related to expected program behavior.
- **Test** – The exercising of software with **test cases**, or a collection of **test cases** (depending on context).

5

Math for Testers

- Graph Theory:
  - What is a graph (directed and undirected)
  - Degree of a node (and In/Out Degrees)
  - Incident Matrix
  - Adjacency Matrix (Symmetric vs Unsymmetrical)
  - Path
  - Connectedness & Condensation
  - **Program Graphs**

6

Math for Testers

- Propositional Logic
- Set Theory:
  - Operations,
  - Relations
- Venn Diagrams
- Functions:
  - Domain,
  - Range,
  - onto,
  - into,
  - one-to-one,
  - many-to-one

7

Symbolic Execution

- Concrete vs Symbolic execution
- Limitations of Symbolic Execution
- Is symbolic execution an “exhaustive” testing technique? Why or why not?
- Symbolic execution trees

8

Symbolic Execution

- Can you symbolically execute this code:

```
if(a > b)
        a = b;
if(b > a)
        b = a;
if(a == b)
        return a;
return -1;
```

- Is anything never executed? Always executed?
- What are the pros/cons of symbolic execution?

9

Boundary Value Testing

- Four kinds of boundary value testing:
  - **Normal** boundary value testing
  - **Robust** boundary value testing
  - **Worst case** boundary value testing
  - **Robust worst case** boundary value testing

10

Boundary Value Testing

- Code or Spec Based?
- Rationale for testing boundaries
- How do we choose?
- Output boundary value testing

11

How to choose?

- Two considerations apply to boundary value testing:
  - Are invalid values an issue?
  - Can we make the “single fault assumption” of reliability theory?
- Consequences of these considerations:
  - Invalid values require the robust choice
  - Multiplicity of faults requires worst case testing

	Valid Values	Valid and Invalid Values
Single Fault	Boundary Value	Robust Boundary Value
Multiple Fault	Worst Case Boundary Value	Robust Worst Case Boundary Value

12

Pros and Cons of Boundary Value Testing

- Advantages
  - Commercial tool support available
  - Easy to do/automate
  - Appropriate for calculation-intensive applications with variables that represent physical quantities (e.g., have units, such as meters, degrees, kilograms)
- Disadvantages
  - Inevitable potential for both gaps and redundancies
  - The gaps and redundancies can never be identified (specification-based)
  - Does not scale up well
  - Tools only generate inputs, user must generate expected outputs.

13

Equivalence Class Testing

- Code or Spec Based?
- Equivalence Relationships
- Equivalence Partitions, and partitioning:
  - Works best when F is a many-to-one function
  - Test cases are formed by selecting one value from each equivalence class.
  - Identifying the classes may be hard
- When to use equivalence class testing:
  - Variables represent logical (rather than physical) quantities.
  - Variables “support” useful equivalence classes.
  - Distinct modes or sets of discrete behavior.

14

Forms of Equivalence Class Testing

- **"Traditional":** focus on invalid inputs

Attributes

- **Normal**: classes of valid values of inputs
- **Robust**: classes of valid and invalid values of inputs
- **Weak**: (single fault assumption) one from each class
- **Strong**: (multiple fault assumption) one from each class in Cartesian Product

15

How to choose an Equivalence Class method?

	Valid Values	Valid and Invalid Values
Single Fault	Weak Normal	Weak Robust
Multiple Fault	Strong Normal	Strong Robust

16

Path Based Testing

- Code or Spec Based?
- DD Paths
- Graph metrics: G_{node}, G_{edge}
- How do we test a loop? (i.e., Huang's Theorem)
- Coverage based testing
- Basis Path Testing

17

Program Graphs: In Class Exercise

```
 -  // Method returns the greatest common divisor of a and b
 -
 -  private static int gcd (int a, int b) {
 -
 1     if(a < 0) {
 2        a = Integer.MIN_VALUE;
 3     } else if(b < 0) {
 4        a = Integer.MIN_VALUE;
 5     } else {
 6        while (b != 0) {
 7           int temp = b;
 8           b = a % b;
 9           a = temp;
 -        }
 -     }
 -
10     return a;
 -  }
```

18

Coverage Based Testing

Design enough test cases such that:

- **Line coverage**: every line in a program is executed at least once.
- **Branch coverage**: each decision/branch has a *true* and *false* outcome at least once.
- **Condition coverage**: each condition in a decision/branch takes all possible outcomes at least once.
- **Branch/Condition coverage**: each condition in a decision takes on all possible outcomes at least once, *and* each decision takes on all possible outcomes at least once.

19

Coverage Based Testing

Design enough test cases such that:

- **Multiple condition coverage**: all possible combinations of condition outcomes in each decision are invoked at least once.
- **MC/DC**: MCC *and* each condition is show to independently affect the outcome of the decision.
- **Path coverage**: all execution paths are executed at least once:
  - A path is a unique sequence of branches from entry to exit.
  - Loops introduce an unbounded number of paths making path coverage criterion not practical.
  - Some paths are impossible to exercise due to relationships of data.
  - The number of executions of a loop may depend on the input variables and hence may not be possible to determine.

20

Decision Table Based Testing

- What are decision tables: Conditionals, Rules, Actions
- Rule Condensation
- “Don’t Care” conditionals
- Rule Counting (even w/ “Don’t Care” conditionals)
- Extended Entry
- Impossible Rules
- Rules to Tests

21

MCDC Testing

- Condition
- Coupled Condition (Strong & Weak)
- Masking Condition
- MCDC Coverage & Independent Effect
- Finding Independent Effect
- Masking for Independent Effect

22

Not Included

- Data Flow Testing
- Unit Testing Retrospective

23

Questions?

24